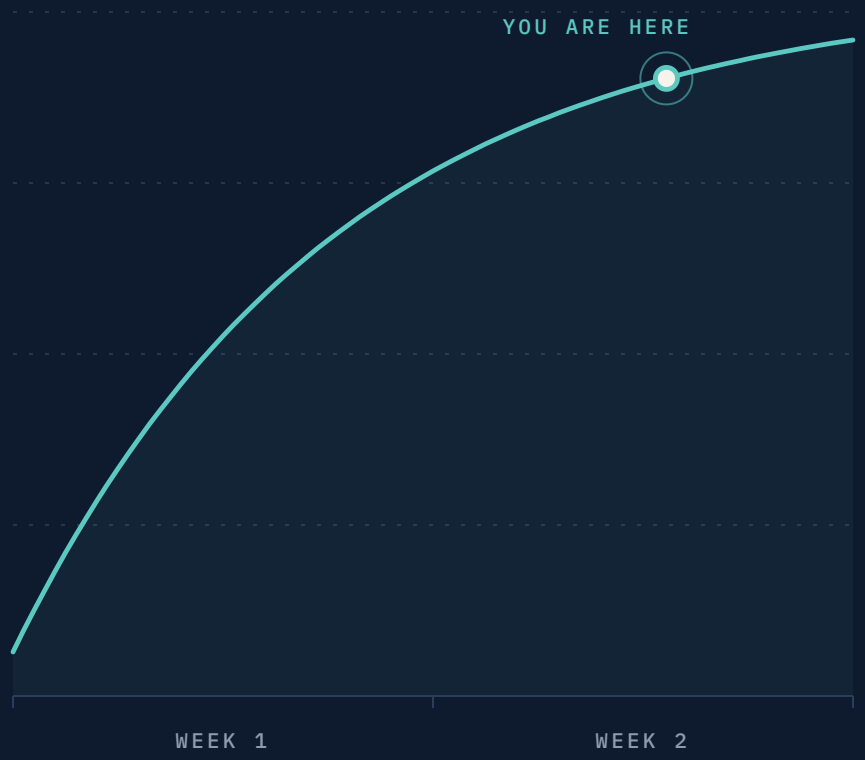
WEEK 2 · DAY 3 · OVERVIEW

Lag Features and Gradient Boosting



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Days 5 and 6 gave a model one input, the series' own past, in one fixed shape.

A desk's next idea is to hand a flexible model everything at once: recent changes, their averages and standard deviations, the other series, the weekday.

Today builds those inputs from earlier rows only, scores Course 5's LightGBM walk-forward against Day 2's bar, plants the classic leak and catches it, tunes the settings, and points the same inputs at the size of a change.

Every output is a value for a date inside the file from earlier dates, never a forecast of rates or spreads.

OBJECTIVES

Build lag, rolling, cross-series and calendar features from earlier rows, and prove each one with the cut test.

Fit **`LGBMRegressor`**, score it walk-forward against the last value, plant a leak and find it, and tune four settings with stopping rows at the end of each training window.

Compare LightGBM with Days 2, 5 and 6's rules on five series, read permutation importance against planted noise columns, and read partial dependence on the size of a change.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 3

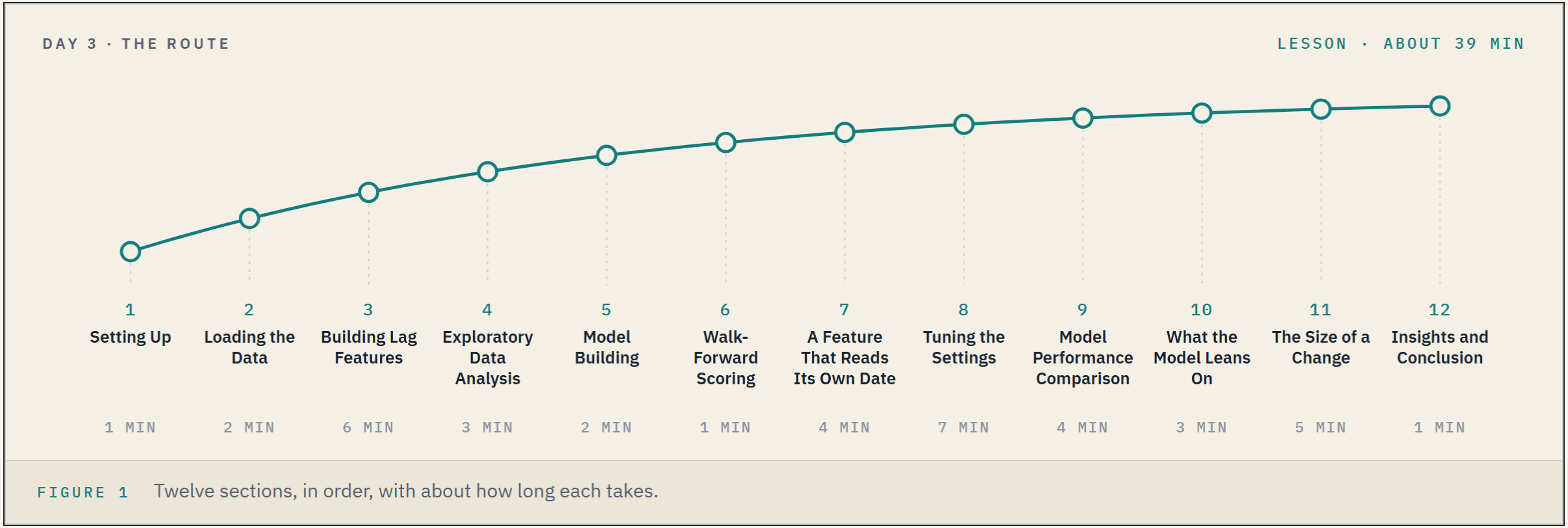


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 39 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import lightgbm as lgb
from lightgbm import LGBMRegressor
from sklearn.inspection import permutation_importance, PartialDependenceDisplay
```

Loading the Data

DAY 2'S FIVE SERIES, IN BP

```
levels = pd.DataFrame({"2 Yr": (development["2 Yr"] * 100).round(2),
                       "10 Yr": (development["10 Yr"] * 100).round(2),
                       "2s10s": ((development["10 Yr"] - development["2 Yr"]) * 100).round(2),
                       "IG": spreads.loc[:CUT, "ig_spread_bp"],
                       "HY": spreads.loc[:CUT, "hy_spread_bp"]})
changes = levels.diff().dropna()
print(len(levels), "levels and", len(changes), "changes, from", changes.index[0].date(), "to", changes.index[-1].date())
```

THE BAR TO BEAT

```
folds = list(TimeSeriesSplit(n_splits=5, test_size=252).split(changes))
bar = pd.Series([changes["10 Yr"].iloc[test].abs().mean() for train, test in folds], index=range(1, 6))
print(bar.round(3).to_dict(), "| average:", round(bar.mean(), 3), "bp | range:", round(bar.max() - bar.min(), 3), "bp")
```

```
{1: 2.69, 2: 3.444, 3: 3.663, 4: 3.306, 5: 6.258} | average: 3.872 bp | range: 3.567 bp
```

Today's model must beat the last value's 3.872 bp, and by Day 2's rule a gain counts only if it is larger than the folds' range of 3.567 bp.

Building Lag Features

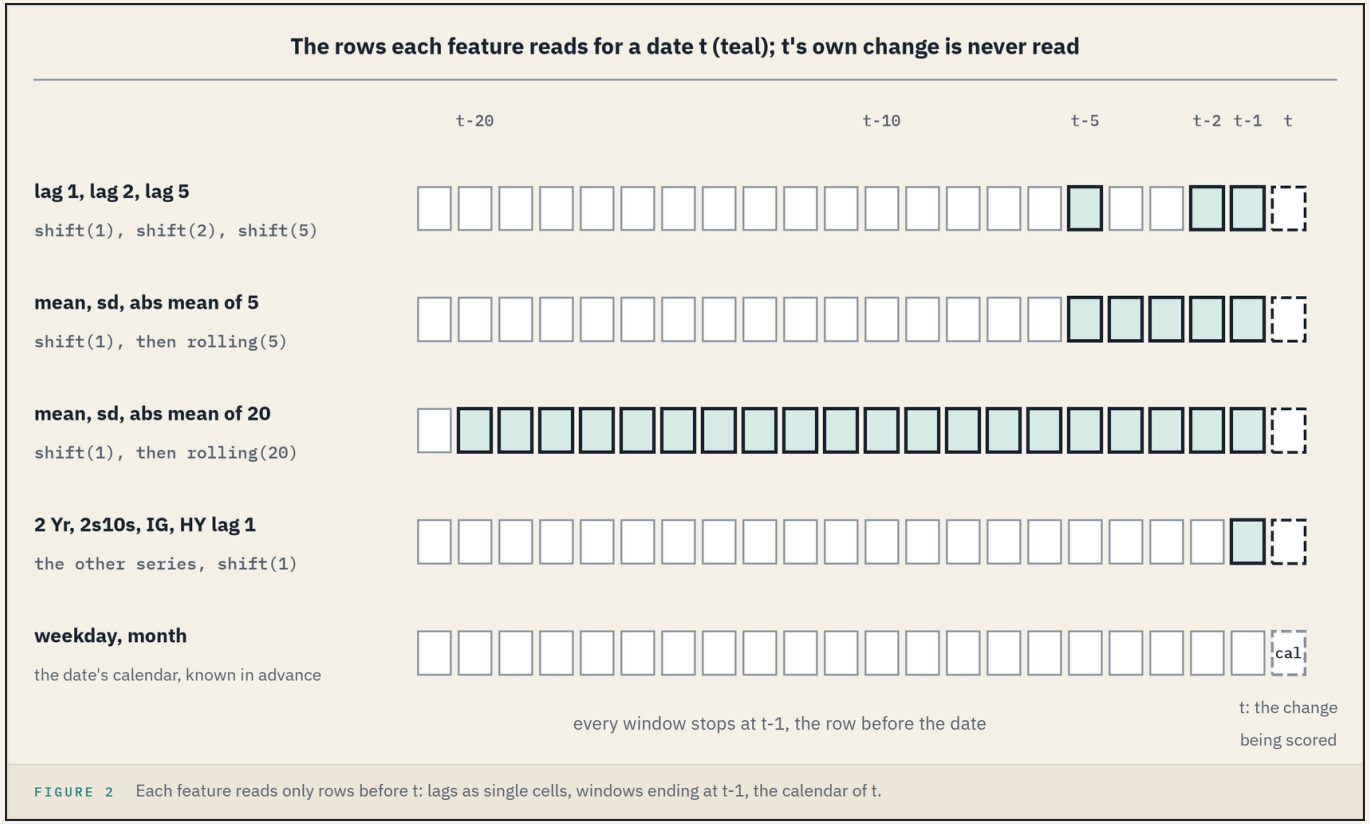


FIGURE 2 Each feature reads only rows before t: lags as single cells, windows ending at t-1, the calendar of t.

SHIFT FIRST, THEN ROLL

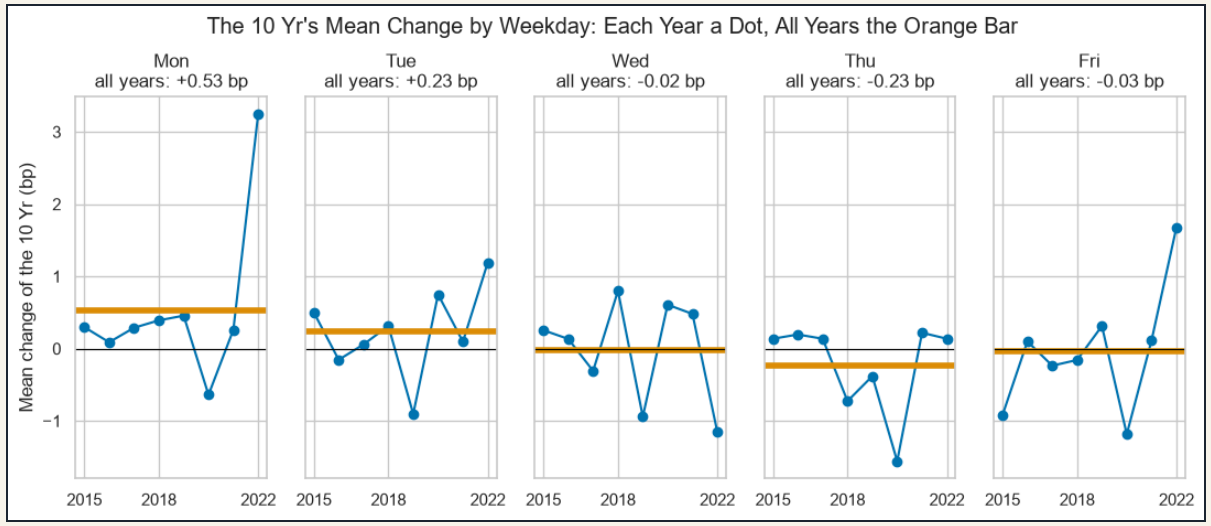
```
past = target.shift(1)
```

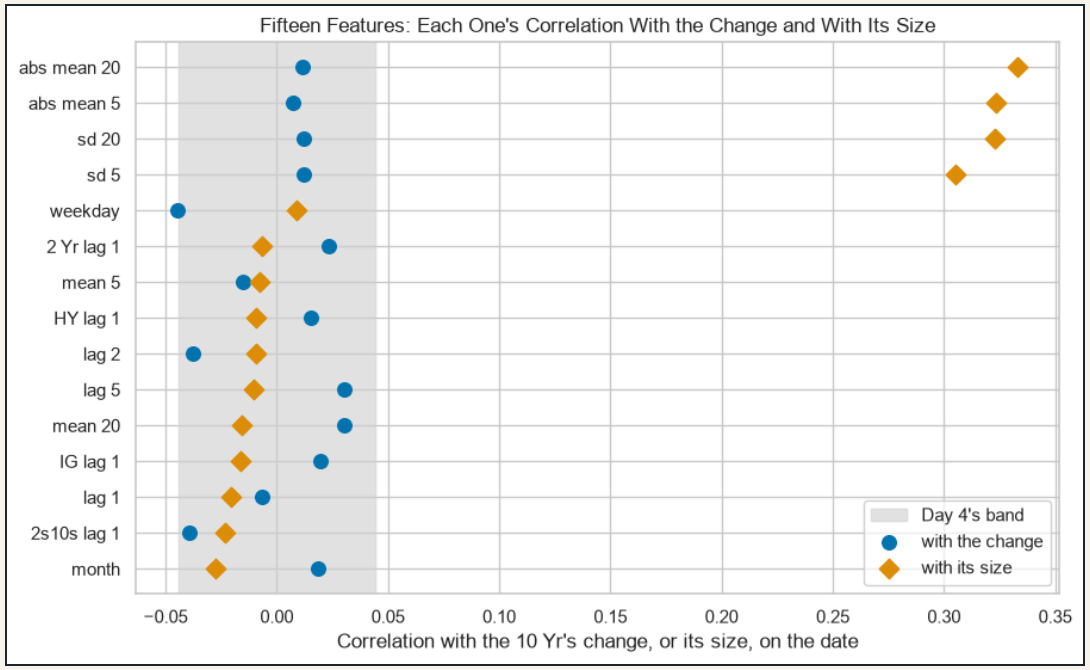
THE CUT TEST ON ALL FIFTEEN

```
15 of 15 columns equal | 0 of 1249 dates moved
```

Fifteen features, each built from earlier rows only, and proved so.

Exploratory Data Analysis





A straight line sees almost nothing in the change; a tree model may find shapes a line cannot, so we fit one.

Fit LightGBM on fold 1 and score it.

Model Building

FIT ON FOLD 1

```
train, test = folds[0]
model = LGBMRegressor(random_state=SEED, n_jobs=1, verbose=-1)
model.fit(X.iloc[train], y.iloc[train])
```

THE QUESTION

Q. What is LightGBM's MAE on those test dates, in bp? Write down a number.

THE THREE LINES

`LGBMRegressor` is Course 5's Day 9 LightGBM for a number instead of a class, with the same settings.

THE SAME MODEL ON FOLD 1'S TEST DATES

```
fitted = pd.Series(model.predict(X.iloc[test]), index=X.index[test])
```

Walk the defaults over all five folds.

Walk-Forward Scoring

THE LOOP AS A FUNCTION

```
def walk_forward_lgbm(X, y, **settings):
    """Fit once a fold on the training rows; keep the MAE in bp and the fitted change on the test dates."""
    maes, values = [], []
    for train, test in folds:
        model = LGBMRegressor(random_state=SEED, n_jobs=1, verbose=-1, **settings).fit(X.iloc[train], y.iloc[train])
        values.append(pd.Series(model.predict(X.iloc[test]), index=X.index[test]))
        maes.append(mean_absolute_error(y.iloc[test], values[-1]))
    return pd.Series(maes, index=range(1, 6)), pd.concat(values)
```

THE DEFAULTS ON ALL FIVE FOLDS

```
defaults, default_values = walk_forward_lgbm(X, y)
```

THE SETTINGS

`**settings` collects any named settings and passes them on to **`LGBMRegressor`** (Day 7).

A Feature That Reads Its Own Date

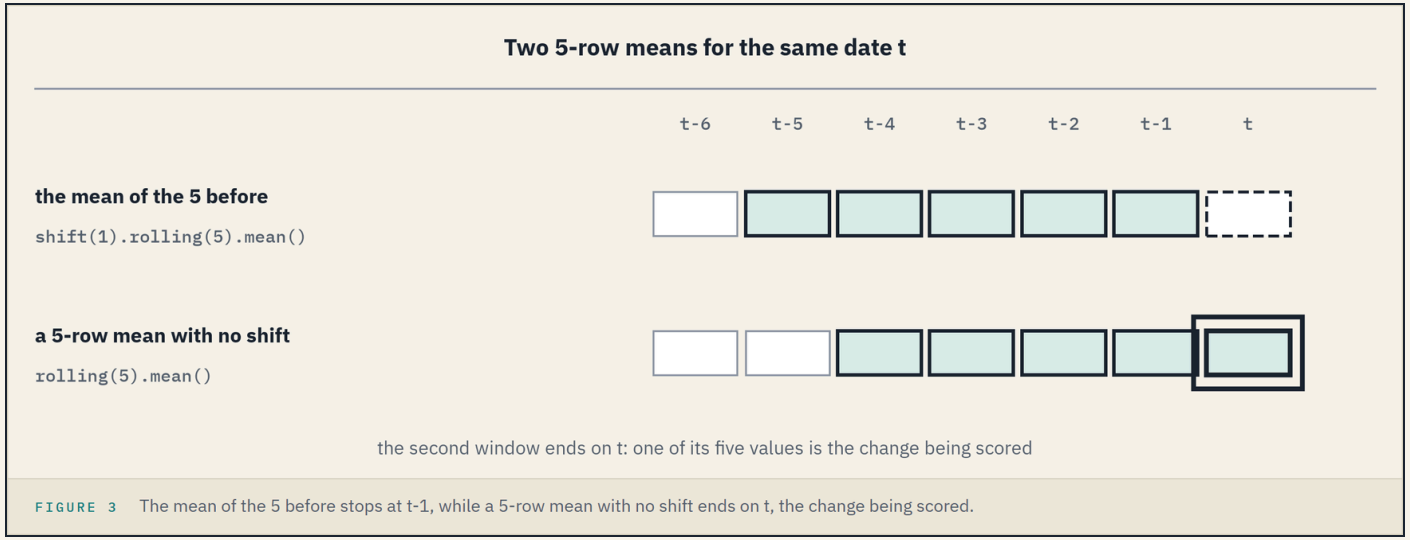


FIGURE 3 The mean of the 5 before stops at t-1, while a 5-row mean with no shift ends on t, the change being scored.

ONE MORE COLUMN

```
X_leak = X.copy()
X_leak["mean 5 with t"] = target.rolling(5).mean()
```

DAY 2'S CUT TEST

```
1 of 1249 values moved, on 2019-12-31
```

Course 3's trailing mean, with no shift, is the classic slip: its window ends on the date it describes.

Tuning the Settings

FOUR SETTINGS

- **learning_rate**: how far each tree moves the fitted value
- **n_estimators**: how many trees
- **num_leaves**: the most leaves in one tree
- **min_child_samples**: the fewest training rows in a leaf

THE QUESTION

Q. Sketch the average fold MAE against the number of trees, 25 to 400, at learning rates 0.01 and 0.3, with 3.872 drawn across.

LEARNING RATE BY TREES

```
for rate in [0.01, 0.03, 0.1, 0.3]:
    for trees in [25, 50, 100, 200, 400]:
        sweep[rate, trees] = walk_forward_lgbm(X, y, learning_rate=rate, n_estimators=trees)[0]
```

THE STOPPING ROWS

```
for fit_rows, test_rows in folds:
    stopper = LGBMRegressor(n_estimators=1000, learning_rate=0.1, random_state=SEED, n_jobs=1, verbose=-1)
```

Score six rules on five series with MASE.

Model Performance Comparison

THE SIX RULES

Day 2's last value and drift, Day 5's AR(1) through `.append()`, and Day 6's simple smoothing with alpha fitted a fold, beside LightGBM's defaults and the chosen setting, on all five series.

EVERY RULE ON EVERY SERIES

```
scores = {}
for name in levels.columns:
    table = past_rules(name)
    features_of, change_of = lag_features(changes, name).loc[X.index], changes[name][X.index]
    table["LightGBM defaults"] = walk_forward_lgbm(features_of, change_of)[0]
    table["LightGBM chosen"] = walk_forward_lgbm(features_of, change_of, **chosen)[0]
    scores[name] = table
```

The settings were chosen on the 10 Yr and are reused as they are.

Shuffle each column against five noise columns.

What the Model Leans On

FIVE NOISE COLUMNS

```
rng = np.random.default_rng(SEED)
X_noise = X.copy()
for k in range(1, 6):
    X_noise[f"noise {k}"] = rng.standard_normal(len(X))
```

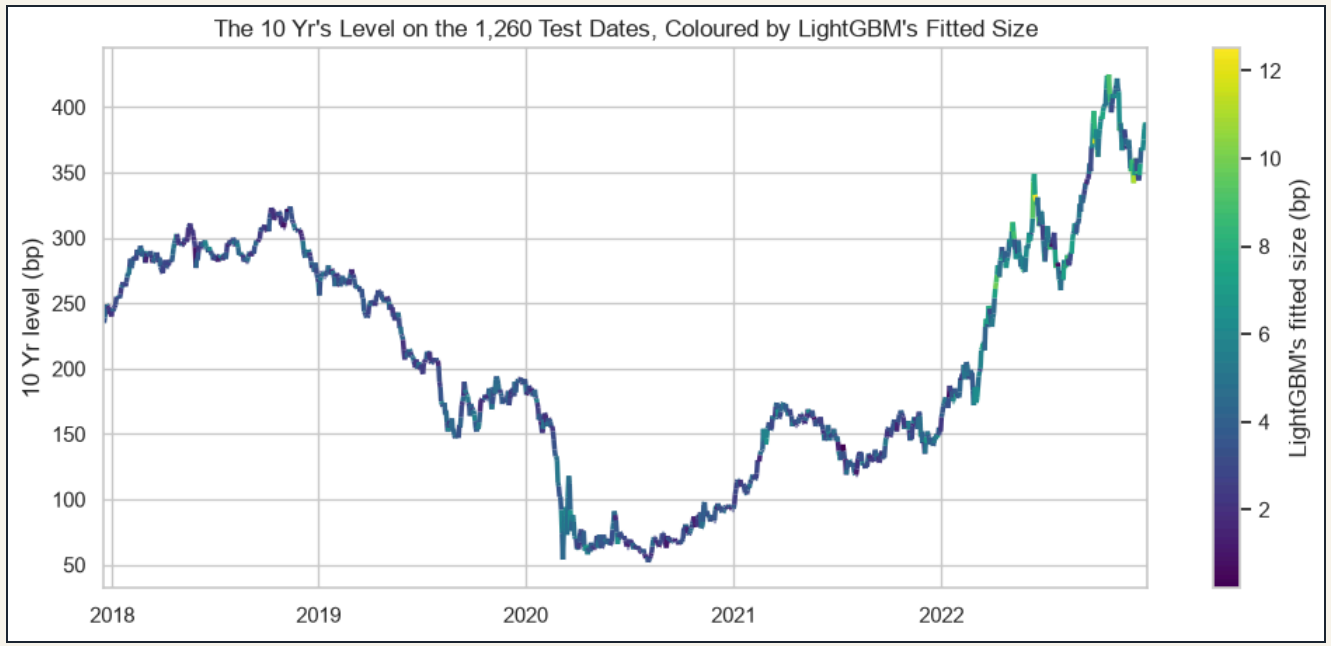
THE QUESTION

Q. Shuffled on the test dates, how many of the 15 features raise the MAE by more than the highest of the five noise columns? Write down a number.

SHUFFLE EACH COLUMN OF THE TEST ROWS

```
rises = {}
for fold, (fit_rows, test_rows) in enumerate(folds, start=1):
    shuffled = LGBMRegressor(random_state=SEED, n_jobs=1, verbose=-1).fit(X_noise.iloc[fit_rows], y.iloc[fit_rows])
    result = permutation_importance(shuffled, X_noise.iloc[test_rows], y.iloc[test_rows], scoring="neg_mean_absolute_error",
                                    n_repeats=10, random_state=SEED, n_jobs=1)
    rises[fold] = pd.Series(result.importances_mean, index=X_noise.columns)
rises = pd.DataFrame(rises)
```

The Size of a Change



COLOUR A LINE BY A THIRD NUMBER

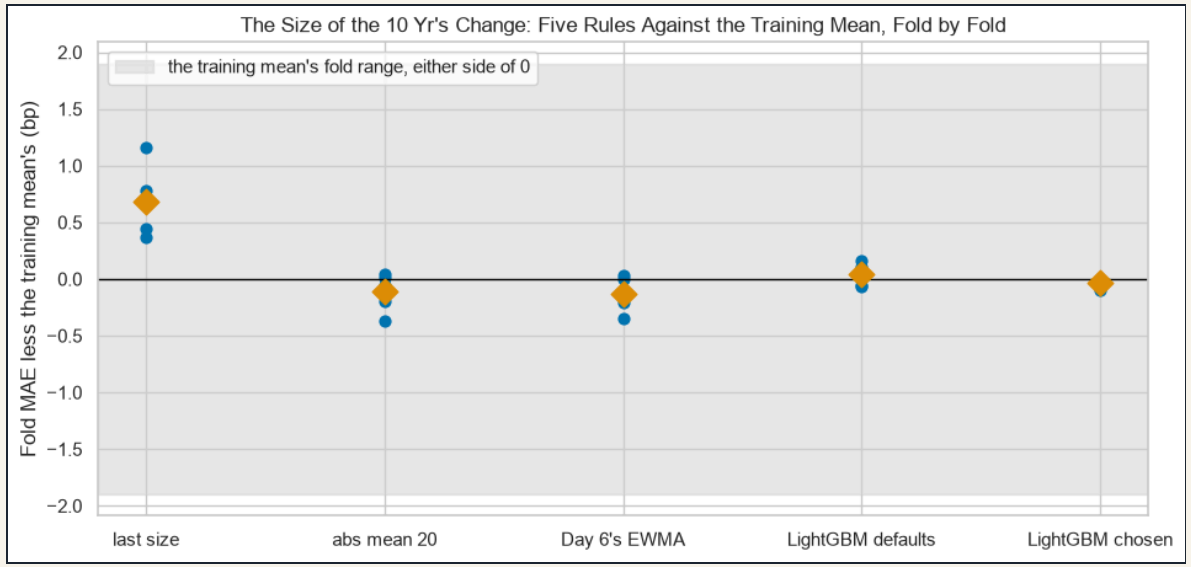
A **colour-graded line** colours each piece of a line by a third number.

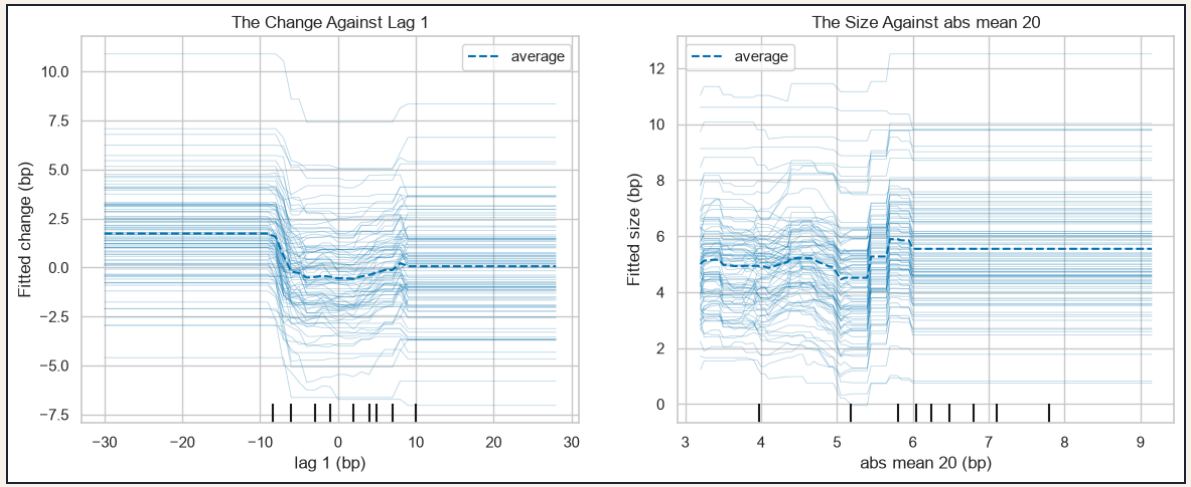
THE SIZE, WALKED FORWARD

```
size = y.abs()
size_defaults, size_values = walk_forward_lgbm(X, size)
```

The fitted size is brightest through 2022 and lifts in March 2020, the stretches where the level moved fastest.

Insights and Conclusion





On the size, every rule from the past sits within the folds' disagreement of the training mean, and LightGBM adds nothing a 20-row average does not hold.

You Can Now

- Build lag, rolling, cross-series and calendar features that stop at the row before the date.
- Prove every feature with the cut test, and catch a feature that reads its own date.
- Fit **`LGBMRegressor`** and score it walk-forward against the last value carried forward.
- Tune the learning rate, trees, leaves and rows per leaf on the folds, with stopping rows at the end of each training window.
- Read permutation importance against planted noise columns, and partial dependence with ICE lines.

NEXT
WEEK 2 · DAY 4

Two Series: Cointegration and Mean Reversion

Day 9 brings two series together: cointegration and a hedge ratio.